

Assignment: The Enchanted Highlands Ascent

Python Programming - Week 11

1 The Story

Fresh from the success of the Royal Expedition, the King has a new quest for his Court Coder. Far beyond the Frozen Peaks lie the **Enchanted Highlands** — a vast land of rolling terrain best described as an $m \times n$ grid, where every cell records the *elevation* of that patch of land.

The King's Royal Cartographer has been tasked with charting the **longest ascending trail** through this terrain. Starting from any cell, the cartographer may step to any of its four neighbours (*up*, *down*, *left*, or *right*), but each step must lead to a cell of **strictly greater elevation**. Diagonal movement is forbidden, and the cartographer must not step outside the boundaries of the map.

As the Royal Court Coder, you must determine the length of the longest such ascending trail that exists anywhere on the map.

2 Problem Statement

Given an integer matrix of dimensions $m \times n$, find the **length of the longest strictly increasing path**. From each cell you may move in one of four directions (left, right, up, down). You may not move diagonally, and wrap-around is not allowed.

The length of a path is the number of cells it visits (a single cell on its own has length 1).

To prove your logic is correct, the King requires:

1. The **length** of the longest increasing path.

3 Technical Constraints

- $1 \leq m, n \leq 200$ (grid dimensions).
- $0 \leq \text{matrix}[i][j] \leq 2^{31} - 1$ (elevation values).
- **Time Complexity:** $O(m \times n)$.
- **Space Complexity:** $O(m \times n)$ to store the memoization table.

Hint: A naive DFS explodes exponentially. The elegant solution is DFS combined with **memoization** — once you know the longest increasing path starting at a cell, you never recompute it.

4 Input & Output Format

Input

1. Line 1: Two space-separated integers m and n (rows and columns).
2. Next m lines: Each line contains n space-separated integers representing a row of the matrix.

Output

- Line 1: A single integer — the length of the longest strictly increasing path.

Example 1

Input:

- $m = 3, n = 3$
- Row 1: 9 9 4
- Row 2: 6 6 8
- Row 3: 2 1 1

```
3 3
9 9 4
6 6 8
2 1 1
```

Output:

```
4
```

Explanation: The longest increasing path is $[1, 2, 6, 9]$, found by stepping from the 1 in the bottom row upward and around the grid.

Example 2

Input:

```
3 3
3 4 5
3 2 6
2 2 1
```

Output:

```
4
```

Explanation: The longest increasing path is $[3, 4, 5, 6]$. Note that although $1 \rightarrow 2 \rightarrow 6$ looks tempting, but moving diagonally is not allowed.

Example 3

Input:

```
1 1
1
```

Output:

1

Explanation: A single cell forms a path of length 1.

5 Implementation Skeleton

Submit a file named `week11_[yourRollnumber].py`.

```
1 """
2 Name           : [Your Name]
3 Roll Number    : [Your Roll Number]
4 Time Complexity Analysis:
5     Explain why the complexity is  $O(m * n)$ .
6     Hint: each cell's result is computed exactly once thanks
7     to memoization; DFS from a cell visits at most 4 neighbours.
8 """
9
10 import sys
11 sys.setrecursionlimit(100000)
12
13 def longest_increasing_path(m, n, matrix):
14     if m == 0 or n == 0:
15         return 0
16
17     # Memoization table: memo[i][j] = length of longest increasing
18     # path STARTING at cell (i, j).
19     memo = [[0 for _ in range(n)] for _ in range(m)]
20
21     directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
22
23     def dfs(i, j):
24         # TODO: If already computed, return memo[i][j]
25         # TODO: Explore all 4 neighbours; only recurse into cells
26         #         whose value is strictly greater than matrix[i][j]
27         # TODO: Store the result in memo[i][j] and return it
28         pass
29
30     longest = 0
31     # TODO: Call dfs(i, j) for every cell and track the maximum
32
33     return longest
34
35
36 if __name__ == "__main__":
37     # Input Handling
38     try:
39         m, n = map(int, input().split())
40         matrix = []
41         for _ in range(m):
42             row = list(map(int, input().split()))
43             matrix.append(row)
44
45         result = longest_increasing_path(m, n, matrix)
46         print(result)
47     except EOFError:
48         pass
```

6 Submission Instructions

- **File Name:** `week11_[yourRollnumber].py`
- **Requirement:** You must include the Time Complexity analysis in the header comment, explaining *why* your solution runs in $O(m \times n)$ time.
- **Note:** Any correct implementation that respects the stated time and space bounds is acceptable. Solutions whose worst-case complexity exceeds $O(m \times n)$ (e.g., plain DFS without memoization) will not receive full credit, even if they produce the correct answer on small inputs.